
PRÁCTICA 1: “ANÁLISIS DE PRESTACIONES”

Arquitectura e Ingeniería de Computadores
E.T.S. de Ingeniería Informática (ETSINF)
Dpto. de Informática de Sistemas y Computadores (DISCA)

Objetivos:

- Calcular la fracción de tiempo de un componente de un programa aplicando la ley de Amdahl.
- Evaluar y comparar las prestaciones de distintas arquitecturas.

Desarrollo:

Ley de Amdahl.

La ley de Amdahl nos indica que si una aplicación utiliza un cierto componente durante una fracción F de su tiempo de ejecución, la aceleración que se obtendría si se mejorara en un factor S dicho componente viene dada por la siguiente expresión:

$$S' = \frac{1}{1 - F + \frac{F}{S}}$$

Sin embargo, en muchas ocasiones, resulta difícil determinar la fracción F , bien por no disponer del código fuente de las aplicaciones, bien por estar distribuida la utilización del componente a lo largo de todo el tiempo de ejecución del programa. Pues bien, la aplicación “a la inversa” de la Ley de Amdahl permite fácilmente obtener dicha fracción.

Para ello, se necesita ejecutar la aplicación bajo estudio con dos tipos del componente en cuestión cuya mejora local S sea conocida. El cociente de los dos tiempos de ejecución obtenidos es la aceleración global S' . Conocidos S y S' , no hay más que despejar F .

Para ilustrar todo esto con un ejemplo, supongamos que se pretende obtener la fracción F del tiempo de ejecución que una aplicación, que realiza operaciones con matrices (`matrix`), emplea en realizar productos escalares. El producto escalar de dos vectores $A = \{a_1, a_2, \dots, a_n\}$ y $B = \{b_1, b_2, \dots, b_n\}$ de tamaño n se define como,

$$A \cdot B = \{a_1, a_2, \dots, a_n\} \cdot \{b_1, b_2, \dots, b_n\} = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

y es una operación básica, por ejemplo, en la multiplicación de matrices. La idea será realizar varias implementaciones de esta operación (el “componente” bajo estudio), en un caso haciendo uso de las instrucciones estándar del procesador y otros casos empleando instrucciones SIMD implementadas por las extensiones SSE, AVX ó AVX512 del procesador del laboratorio (Intel i5-11600K de 11ª generación, microarquitectura Sunny Cove). **Hay que recordar que todas estas extensiones pueden no existir en procesadores más antiguos.**

Cálculo de la aceleración local S

Necesitamos, por tanto, disponer de las implementaciones del componente en cuestión. Las funciones *Scalar()*, *ScalarSSE()*, *ScalarAVX()*, *ScalarAVX512()* implementan el producto escalar de dos vectores tal y como se ha comentado en el apartado anterior. La implementación con instrucciones SIMD debe ser más rápida ya que dichas instrucciones permiten operar simultáneamente con varios datos, (4 valores de coma flotantes de precisión simple en SSE, 8 en AVX y 16 en AVX512). En las figuras 1, 2, 3 y 4 pueden verse las implementaciones con el juego de instrucciones estandar y el SSE.

La relación entre la velocidad obtenida por la versión estandar y cada una de las SIMD será S en la expresión de la Ley de Amdahl.

Para obtener esta relación utilizaremos un sencillo programa en lenguaje C que realiza productos escalares dentro de un bucle. La figura 5 muestra el código del programa, denominado `scalar.c`.

Viendo el código vemos que unas directivas macro controlan la versión de la función *Scalar()* que será compilada y por tanto utilizada. Con las diferentes versiones obtendremos diferentes tiempos de ejecución y así podremos estimar los valor de S en cada caso. Será necesario obtener el tiempo de ejecución para la versión estandar (t_{std}), las versiones SIMD (t_{sse} , t_{avx} , t_{avx512}) y la versión eliminando el producto escalar para obtener la sobrecarga del bucle de medida y la inicialización de los vectores (t_{load}). Restando este valor a los anteriores podremos obtener el tiempo real empleado por el componente que nos interesa.

Para obtener los diferentes tiempos de ejecución, generaremos las diferentes versiones del ejecutable compilando desde la consola de linux. Para ello pasaremos al compilador en cada caso la macro (`-Dxxx`) correspondiente y la opción (`-mxxx`) que activa dicha extensión SIMD en el compilador. **Por defecto estas extensiones no están activas en el compilador para evitar problemas de compatibilidad con procesadores más antiguos.**

```
gcc -O0 -o scalar-std scalar.c -DSCALAR
gcc -O0 -o scalar-sse scalar.c -DSSE -msse
gcc -O0 -o scalar-avx scalar.c -DAVX -mavx
gcc -O0 -o scalar-avx512 scalar.c -DAVX512 -mavx512f
gcc -O0 -o scalar-load scalar.c -DLOAD
```

los ejecutaremos y mediremos los tiempo usando la orden `time`, pero **ANTES LEE EL PÁRRAFO SIGUIENTE:**

ESTAS INSTRUCCIONES NO SON NECESARIAS SI SE REALIZA LA PRÁCTICA DESDE UN ENTORNO VIRTUAL (Polilabs, VirtualBox, VMware, ...)

Antes de ejecutar el programa tenemos que fijar la velocidad del procesador para que las medidas sean uniformes. Por defecto los cores del procesador están en modo *ondemand* y la velocidad de los mismos varía según la carga. Esto no es adecuado para tomar medidas. Para cambiar el modo de los cores empleamos la orden `cpupower frequency-set`, pudiendo ver el modo en que está con la orden `cpupower frequency-info`. Fijaremos el modo de cada core a *performance*. El parametro `-c #` identifica el core que vamos a fijar:

```
float Scalar(float *s1,
            float *s2,
            int size)
{
    int i;
    float prod = 0.0;

    for(i=0; i<size; i++) {
        prod += s1[i] * s2[i];
    }

    return prod;
} // end Scalar()
```

Figura 1: Producto escalar implementado con instrucciones estandar.

```
float ScalarSSE(float *m1,
               float *m2,
               int size)
{
    float prod = 0.0;
    int i;
    __m128 X, Y, Z;
    Z = _mm_setzero_ps(); /* all to 0.0 */
    for(i=0; i<size; i+=4) {
        X = _mm_load_ps(&m1[i]);
        Y = _mm_load_ps(&m2[i]);
        X = _mm_mul_ps(X, Y);
        Z = _mm_add_ps(X, Z);
    }
    for(i=0; i<4; i++) {
        prod += Z[i];
    }
    return prod;
} // end ScalarSSE()
```

Figura 2: Con instrucciones SSE.

```
float ScalarAVX(float *m1,
               float *m2,
               int size)
{
    float prod = 0.0;
    int i;
    __m256 X, Y, Z;
    Z = _mm256_setzero_ps(); // all 0.0
    for(i=0; i<size; i+=8) {
        X = _mm256_load_ps(&m1[i]);
        Y = _mm256_load_ps(&m2[i]);
        X = _mm256_mul_ps(X, Y);
        Z = _mm256_add_ps(X, Z);
    }
    for(i=0; i<8; i++) {
        prod += Z[i];
    }
    return prod;
} // end ScalarAVX()
```

Figura 3: Con instrucciones AVX.

```
float ScalarAVX512(float *m1,
                  float *m2,
                  int size)
{
    float prod = 0.0;
    int i;
    __m512 X, Y, Z;
    Z = _mm512_setzero_ps(); // all 0.0
    for(i=0; i<size; i+=16) {
        X = _mm512_loadu_ps(&m1[i]);
        Y = _mm512_loadu_ps(&m2[i]);
        X = _mm512_mul_ps(X, Y);
        Z = _mm512_add_ps(X, Z);
    }
    for(i=0; i<16; i++) {
        prod += Z[i];
    }
    return prod;
} // end ScalarAVX512()
```

Figura 4: Con instrucciones AVX512.

```
int main(int argc, char * argv[]) {

    int      i;
    int      rep=10;
    int      msize=MSIZE;
    float     fvalue;

    if (argc == 2) {
        rep = atoi(argv[1]);
    } else if (argc == 3) {
        rep = atoi(argv[1]);
        msize = atoi(argv[2]);
        msize = (msize > MSIZE) ? MSIZE : msize;
    } // end if/else

    fprintf(stderr, "\n");

    fprintf(stderr, "Rep = %d / size = %d\n", rep, msize);

    for(i=0; i<rep; i++) {
        init_vector(vector_in, msize);
        init_vector(vector_in2, msize);
    #if defined(LOAD)
        // DO NOTHING
    #elif defined(SSE)
        fvalue = ScalarSSE(vector_in2, vector_in, msize);
    #elif defined(AVX)
        fvalue = ScalarAVX(vector_in2, vector_in, msize);
    #elif defined(AVX512)
        fvalue = ScalarAVX512(vector_in2, vector_in, msize);
    #elif defined(SCALAR)
        fvalue = Scalar(vector_in2, vector_in, msize);
    #else
        #error " * * * Instruction set macro unknown! * * * "
    #endif
    } // end for

    exit(0);

} // end main()
```

Figura 5: Programa `scalar.c` empleado para evaluar la relación de las diferentes implementaciones del producto escalar.

```
sudo cpupower -c all frequency-set -g performance # introducir el password
```

lo verificamos con,

```
cpupower -c all frequency-info
```

O,

```
cat /proc/cpuinfo
```

y ya podemos proceder con las ejecuciones.

IMPORTANTE: os recordamos que en linux el directorio actual no está en el PATH, obligándonos a colocar ./ antes de cada programa que queramos ejecutar desde la consola. Para evitar esto hay que ejecutar esta línea en la consola en la que estéis trabajando, o añadirla al final del fichero \$HOME/.bashrc,

```
export PATH=./:$PATH
```

Tal y como hemos dicho ejecutamos las diferente versiones del programa `scalar-xxx`,

```
time scalar-xxx 1000000 1024
```

los parámetros simplemente indican que queremos hacer 1,000,000 repeticiones del bucle y los vectores tendrán un tamaño de 1024. Anotaremos los tiempos empleados por cada tarea (**recordar que hay que sumar sólo *user+system***) como t_{xxx} y estimaremos las S_{xxx} como hemos indicado anteriormente,

$$S_{xxx} = \frac{t_{std} - t_{load}}{t_{xxx} - t_{load}}$$

Con la información obtenida completamos la siguiente tabla,

	t	S
<i>load</i>	7,247	—
<i>std</i>	10,794	—
<i>sse</i>	9,239	1,781
<i>avx</i>	8,286	3,414
<i>avx512</i>	9,170	1,844

⇒ ¿Qué podemos observar? ¿Hay diferencias de aceleración entre las diferente versiones? ¿Se ajustan a lo esperado? ¿Cuál sería la implementación con mejores prestaciones?

Para experimentar con la ley de Amdahl vamos a elegir la versión estandar (*std*) y la versión SIMD con mejores prestaciones (*best*), y las vamos a emplear en la aplicación `matrix`. Esta aplicación calcula el producto de matrices y para ello emplea el producto escalar en repetidas ocasiones.

Mediremos el tiempo de ejecución de dicha aplicación con la versión estandar y la versión SIMD elegida. La relación entre los tiempos de ejecución será la aceleración global que la aplicación alcanza al emplear una versión mejorada del producto escalar. Esta relación es S' en la Ley de Amdahl. Por lo tanto, sustituyendo S y S' en la ley de Amdahl, es posible despejar F para obtener la fracción del tiempo que la aplicación `matrix` hace uso del producto escalar estandar.

Cálculo de la fracción de tiempo local F

Vamos a aplicar el proceso descrito para obtener la fracción de tiempo que la aplicación `matrix` emplea en realizar productos escalares en su versión estandar.

Necesitaremos recompilar el código fuente de dicha aplicación para poder obtener los tiempos que nos hacen falta,

$$S' = \frac{t_{Mstd}}{t_{Mbest}}$$

en este caso deberemos emplear los siguientes comandos de compilación desde la consola para generar los ejecutables,

```
gcc -O0 -o matrix-std matrix.c -lm -DSCALAR
```

```
gcc -O0 -o matrix-best matrix.c -lm -DYYY -mxxx
```

y obtener t_{Mstd} y t_{Mbest} empleando la orden `time`,

```
time matrix-std 1 1024
```

```
time matrix-best 1 1024
```

en este caso sólo realizaremos una repetición y usaremos matrices también de 1024x1024, para que el tamaño del vector sea el mismo que en las mediciones anteriores. Anotaremos los tiempos, el valor de S' (aceleración global, en este apartado) y S (aceleración local, en apartado anterior),

<i>matrix</i>	<i>t</i>	S'	S
<i>std</i>	4,903	—	—
<i>best</i>	2,154	2,276	1,844

⇒ Con los datos obtenidos, calcula el porcentaje de tiempo F que esta aplicación emplea en realizar productos escalares.

$$F = 1,225$$

Cálculo experimental de la fracción de tiempo local (F_{exp})

Aunque en general esto no es posible, dada la naturaleza del código de `matrix` y del componente que estamos evaluando, es posible obtener experimentalmente la fracción del tiempo que esta aplicación emplea en realizar productos escalares. Para ello simplemente tenemos que recompilar el código con el valor de macro `LOAD` para eliminar los

productos escalares y así poder estimar cuanto tiempo dedica el programa al resto de tareas. A este tiempo lo denominaremos t_{Mres} y nos permitirá calcular la fracción de tiempo experimentalmente (F_{exp}) mediante la siguiente fórmula,

$$F_{exp} = \frac{t_{Mstd} - t_{Mres}}{t_{Mstd}}$$

compilando y ejecutando la nueva versión del programa,

```
gcc -O0 -o matrix-load matrix.c -lm -DLOAD
```

podemos obtener t_{Mres} como hicimos anteriormente.

⇒ Calcula experimentalmente la F_{exp} empleada por el producto escalar en `matrix`. Compárala con la obtenida mediante la ley de Amdahl.

$F_{exp} = 0,778$

Recogida de medidas para análisis de prestaciones.

En este apartado se van a medir las prestaciones de los computadores del laboratorio empleando los siguientes programas:

- dos *benchmarks* sintéticos (**dhrystone**, para aritmética entera, y **whetstone**, para aritmética en coma flotante)
- dos aplicaciones reales: el compilador del lenguaje de programación C **gcc**, que sólo utiliza aritmética entera, y la aplicación **xv** de procesamiento de imagen.

Para ello, ejecutaremos los programas, midiendo los tiempos de ejecución.

- **dhrystone** (100.000.000 iteraciones):

```
time dhrystone
```

Indicarle que debe de realizar 100.000.000 de iteraciones cuando el programa lo pregunte.

Anotaremos el tiempo de ejecución $T_{dhrystone}$ del programa. $T_{dhrystone} = 9,153$

- **whetstone** (100.000 iteraciones):

En este caso teclear:

```
time whetstone 100000
```

Anotaremos el tiempo de ejecución T_{whet-h} . $T_{whetstone} = 1,998$

- Compilador del lenguaje C, compilando una aplicación:

Compilaremos la aplicación **xv**. Para ello, suponiendo que los fuentes están ubicados en la carpeta `xv-310a` de nuestro directorio, teclearemos:

```
cd xv-310a/
make clean
time make
```

Anotar el tiempo de ejecución T_{gcc} . $T_{gcc} = 20,481$

- Aplicación *xv*:

Ejecutar la aplicación *xv* que acabamos de compilar, abriendo un fichero de imagen:

```
cd ..
time xv-310a/xv -wait 5 mundo.jpg
```

Anotar el tiempo de ejecución T_{xv} . $T_{xv} = 1,355$

Análisis de las prestaciones.

Una vez recogidas las medidas procedemos al análisis de prestaciones.

La siguiente tabla muestra los tiempos de ejecución en segundos obtenidos al ejecutar los cuatro aplicaciones bajo estudio en 3 máquinas. En la tabla 1 se asume que disponemos de los resultados obtenidos en dos máquinas hipotéticas A y B. La columna *C* hace referencia a la máquina con la que estamos trabajando en el laboratorio donde pondremos los resultados obtenidos para cada aplicación en las cuatro primeras filas.

Programa/Máquina	<i>A</i>	<i>B</i>	<i>C</i>
dhystone	5	18	9,153
whetstone	2.5	10	1,998
gcc	40	130	20,481
xv	4.5	15	1,355
Media aritmética	13	43	8,247

Tabla 1: Tiempos de ejecución de las aplicaciones y media aritmética

A partir de los tiempos de ejecución medidos para cada aplicación en las tres máquinas procedemos a comparar las prestaciones. Para ello, utilizaremos dos métricas clásicas: la media aritmética de los tiempos de ejecución y la media geométrica de los tiempos de ejecución normalizados a la máquina *B* (véase, la media geométrica de los speedups o aceleraciones de las aplicaciones individuales respecto a la máquina *B*). La media aritmética se incluirá en la última fila de la tabla anterior y la media geométrica en la tabla 2. Debido al gran volumen de datos utilizados para analizar prestaciones con frecuencia se utilizan hojas de cálculo o scripts específicos. Se aconseja utilizar una hoja de cálculo y utilizar la función PROMEDIO (o AVERAGE) para obtener la media aritmética, y MEDIA.GEOM (o GEOMEAN) para la media geométrica.

Programa/Máquina	<i>A</i>	<i>B</i>	<i>C</i>
dhystone	3.6 (18/5)	1	1,996
whetstone	4	1	5,005
gcc	3.25	1	1,953
xv	3.3333	1	11,070
Media geométrica de los speedups respecto a B	3,53412	1	3,833

Tabla 2: Media geométrica de los tiempos de ejecución normalizados a la máquina B. En las celdas de las aplicaciones debe indicarse el speedup o aceleración respecto a B.

Preguntas adicionales:

- ¿Cuál es el speedup (o mejora del sistema) de la máquina A sobre la B si se considera media aritmética? .. $43 / 13 = 3,31$ más rápida
- ¿Cuál es el speedup de la máquina A sobre la B si se considera media geométrica de los speedups? $3,53 / 1 = 3,53$ veces más rápida